WONDR

An AI agent that plans a mystery day for you and your friends, books it, puts it on your calendar, and reveals it one stop at a time.

> Agent Builders Hackathon (Omnara), San Francisco. Judging: how useful it is, how production-ready it is, and whether the live demo works. It must be deployed. Submissions close 5:30 pm on Devpost. Version 3 is rebuilt around the agreed split: you build the web app, your teammate builds the agent, and a small written contract joins the two.

1. The idea

You tell WONDR when you are free, where you are, and what you are into. It builds a full day of real places and activities (a run, a yoga class, brunch, a dance lesson, dinner), books what needs booking, adds it to your calendar and your friends' calendars, and keeps the details secret. You see only a start time and a dress code. Each stop is revealed shortly before you need to go there.

What makes it an agent and not a prompt: it searches the real world, checks opening hours and travel times, makes reservations, and writes to calendars. The web app around it is deliberately simple.

2. Who builds what

You (B): the web app	Teammate (A): the agent
Next.js (TypeScript) app on Vercel, and the deploy Google login with Supabase Auth Profile form saving interests, location and time window to a profiles table Plan my day button that calls the agent Plans table that stores results Timeline screen showing the plan, with locked and revealed stops	Model access through the Omnara credits (lib/llm.ts) Planning agent: finds real venues, validates hours, travel time and budget Dress code, teaser and the order of the day Reservation pipeline with fallbacks The API route that runs the agent and writes the result to the plans table

The handshake between you

- Section 5 is the contract: the two tables, the Plan JSON shape, and one API route. Agree on it in the first 30 minutes and put it in main/docs/contract.md.
- Your teammate writes a mock plan (lib/agent/mockPlan.ts) in the first 30 minutes that follows the contract exactly. You build the whole button-to-timeline flow against that mock.

- You save a sample profile in the database early, so your teammate can run the agent against real input without waiting for the UI.
- When the real agent works, your teammate swaps the mock for it in one place. Nothing in the UI changes.

What the split leaves out, and who picks it up

Your list covers rungs 1 to 3 below. These pieces from the original idea are not on either list yet, so decide now who takes them:

Piece	Suggested owner	Why
Google Calendar events (create at plan time, patch at reveal)	B, after the timeline works	B owns login and tokens, and the calendar scope must be set up in the login anyway (section 8)
Reveal behavior and demo mode (compress a day into 2 minutes)	B	Pure UI logic on the timeline screen
Booking status chips on the timeline	B	Reads booking fields from the plan JSON
Friends: emails on the plan form, added as calendar attendees	B	Cheap version of the group idea; a full join-by-link flow is cut
Reservations, re-plan endpoint	A	Agent work
Email fallback for reservations (Resend)	B if free, otherwise A	Small and self-contained
Demo script, backup video, Devpost text	Both, last hour	

> Honest note: the agent side is the harder one, and reservations are the riskiest piece of the whole project. Plan to finish the base app early, then take calendar, demo mode, the email fallback and anything else that frees your teammate up.

3. Scope ladder

Each rung must work on the live URL before the next starts, so at any moment you have something you can demo.

Rung	What works	Owner
1	App deployed, Google login works, database connected, both of you can push and deploy	B (A: model and search keys working)
2	Profile form saves; Plan my day runs against the mock plan and shows the timeline	B (A: mock plan and route)
3	The real agent replaces the mock: real venues, validated hours and travel times	A
4	Reveal behavior with demo mode; calendar placeholder events created and patched at reveal	B

Rung	What works	Owner
5	Reservations: at least one confirmed booking path with proof, plus fallbacks	A
6	Re-plan (running late, swap) and booking chips on the timeline	A and B
Stretch	Browser agent on a second provider, free/busy suggestions, Telegram, Stripe test mode	Only after rung 6
Cut	Join-by-link groups, photo recap, multi-city, voice calls	

Stripe, in one line: it is a payments platform (cards, payment links, splitting costs between friends). You have credits, but the judging criteria do not mention payments, and payments add failure cases that compete with reservations. If you are ahead before 4:00, a test-mode Checkout link to split the cost is about 30 to 60 minutes.

4. User flow and screens

Step	What happens	Owner
1. Log in	Sign in with Google through Supabase. The login also asks for calendar permission, so nobody has to consent twice.	B
2. Profile	Form saves interests, location and time window (plus dietary needs, dislikes and budget if cheap to add). Saved once, editable later.	B
3. Plan my day	Pick a date, optional vibe, budget cap and friends' emails. A consent line says the agent may book on the user's behalf up to the budget cap. Tap the button.	B
4. Agent works	Screen shows live progress (searching brunch spots, checking opening hours, booking). Takes up to a minute or two.	B shows, A writes
5. Teaser	Timeline shows start time, dress code, vibe line and countdown. Stops appear as locked cards with only their time.	B
6. Reveals	A stop unlocks 45 minutes before it starts: name, address, directions, what to do. The calendar event is patched with the real details.	B
7. Adapt (rung 6)	Running late or swap calls the re-plan endpoint, which changes the rest of the day.	A and B

Screens (four)

- Login
- Profile
- Plan my day (form, then progress)
- Timeline (teaser, locked and revealed stops, booking status, demo mode toggle)

5. The contract

How a plan is made

- Tap Plan my day: the app inserts a row in plans with status planning and a snapshot of the input, then calls POST /api/plan with the plan id.
- The route (your teammate's) checks the user is logged in and owns the row, runs the agent, appends progress messages to the row, and finally writes the result and sets status to ready (or failed with an error message).
- The timeline (yours) listens to that row with Supabase realtime, or polls every two seconds as a fallback, and renders when status is ready.
- The agent can take longer than a web request likes. Set a longer maxDuration on the route and check what your Vercel plan allows; the row-and-status design means the page never waits on a single request.

Tables

Table	Columns	Owner
profiles	user_id (key, linked to the auth user), display_name, city, interests text[], dislikes text[], dietary text[], default_window_start, default_window_end, default_budget	B
plans	id, user_id, status (planning, ready, failed), input jsonb, result jsonb, progress jsonb (list of messages), error, created_at	B creates, A writes result
google_tokens (rung 4)	user_id, refresh_token (encrypted), scopes. Saved at sign-in; see section 8	B
bookings (rung 5, optional)	plan_id, stop_position, method, status, confirmation_code, proof_url, cancel_url. Can also live inside the plan JSON	A

Row-level security: users can read and write only their own profiles and plans. The route uses the server-side key to write results.

Shared types (main/lib/types.ts, written by A first)

```
type PlanInput = {
  date: string;               // "2026-11-01"
  windowStart: string;        // "09:00"
  windowEnd: string;          // "22:00"
  city: string;
  interests: string[];
  dislikes?: string[];  dietary?: string[];
  budgetCap?: number;  vibe?: string;
  friendEmails?: string[];
};

type Stop = {
  position: number;
  title: string;              // "Sunrise yoga"
  category: string;           // "fitness", "food", "culture" ...
  venueName: string;  address: string;
  lat?: number;  lng?: number;
  startsAt: string;  endsAt: string;   // ISO times
  description: string;
  sourceUrl?: string;
  booking: { status: "none" | "pending" | "confirmed" | "walk_in" | "failed";
             method?: "link" | "email" | "browser";
             confirmationCode?: string;  proofUrl?: string };
};

type Plan = { title: string; teaser: string; dressCode: string; date: string; stops: Stop[] };
```

Rules both sides rely on

- A stop is revealed when the current time is at or after startsAt minus 45 minutes. Demo mode swaps the real clock for a fast fake one, so this one rule drives everything.
- For the hackathon the browser receives the whole plan and the UI hides locked stops. That is fine for a demo; a production version would strip hidden fields on the server.
- Every time and date is stored as an ISO string with timezone. Mixing local and UTC times is the most common bug in a day-planner, so agree on this now.

6. Stack

Layer	Choice	Notes
App and API	Next.js (TypeScript) on Vercel, Root Directory set to main	One codebase for UI and the agent route
Database, auth, realtime	Supabase	Google login, Postgres, realtime on plans
Model	Omnara credits	All calls through main/lib/llm.ts, so the provider can change in minutes
Venue search and details	Exa and context.dev	Exa is not on the all-attendee credits list; confirm in Discord (section 11)
Booking	Deep links, email through Resend, browser agent in a Freestyle or Unikraft sandbox	Tiered, see section 9
Calendar	Google Calendar API	Scope requested at login

7. The agent (A)

Tool	Purpose
search_venues	Exa search by category, area and time of day
get_venue_details	context.dev scrape: hours, price, booking platform, contact email
check_travel_time	Travel time between consecutive stops
save_progress	Appends a message to plans.progress so the UI can show it
make_booking	Runs the booking pipeline for one stop (section 9)
replan	Rebuilds the rest of the day from a given stop and reason (rung 6)

Planning loop

- Read the input and the profile. Draft a skeleton with an energy curve (active morning, food midday, creative afternoon, social evening).
- For each slot, search real venues, then validate: open at that time, reachable from the previous stop, within budget. Anything that fails validation is replaced, not kept.
- Choose a dress code that fits every stop, write a one-line teaser, and return a Plan that matches the types in section 5.
- Then book the stops that need it. If a booking fails, swap in a venue that takes walk-ins and mark it so.
- Return structured JSON only and validate it against the Plan type before writing it. A malformed plan must never reach the UI.

8. Calendar sync (B, rung 4)

- At plan time, create one placeholder event per stop titled Mystery stop N, with the time window and dress code. Friends' emails from the form are added as attendees.
- At reveal time, patch each event with the real title, location and notes. Re-plan updates or deletes events.
- Set it up from the start, since you are building the login: it must request the calendar.events scope and offline access with a consent prompt. If login ships without it, every user has to sign in again later.
- Supabase gives you the Google access and refresh tokens only in the sign-in session. Save the refresh token to google_tokens right then, in the auth callback, encrypted. It will not be available later.
- Keep the Google Cloud OAuth app in Testing mode and add both of you and any demo accounts as test users. Verification review cannot be done in a day.
- Where the reveal patch runs: the simplest version patches when the client sees a stop become revealed (works in demo mode too). A scheduled function every minute is the sturdier option if there is time.
- Fallback for people without Google: download an .ics file with the same events.

9. Reservations (A, rung 5)

Most reservation platforms have no public booking API, so build in tiers and always keep a fallback. Order: 1, 3, 5 first, then try 2 on a single platform.

Tier	Method	Risk
1	Prefilled deep link or official booking where the venue's platform supports it	Lowest; may still need a user tap
2	Browser agent in a sandbox (Playwright plus model vision) on one platform	Bot checks and layout changes; stop and fall back if blocked
3	Reservation request by email (Resend)	Not instant; confirmation arrives later
4	Voice AI call (stretch)	Setup time
5	Walk-in swap: replace the stop with a venue that takes walk-ins	None; always available

- Book only after the user has ticked the consent line with a budget cap. Prefer free cancellation. Do not submit a step that needs a payment card unless a deposit is explicitly allowed.
- Never try to get past a CAPTCHA or bot check. Hand it to the next tier.
- Idempotent: check for an existing booking before booking, so a retry never double books. Keep a dry-run mode for development.
- Save proof (confirmation code and screenshot) in the stop's booking object. The timeline shows it as a status chip, and it doubles as demo evidence.
- Test on two or three real venues as soon as the real plan works, then constrain the demo plan to venues that worked.

10. Repository and working rules

```
wondr/
  README.md   .gitignore
  main/
    app/                      pages, components                        (B)
    app/api/plan/route.ts     runs the agent, writes the result        (A)
    lib/types.ts              shared types                             (A writes, both use)
    lib/llm.ts                the only file that calls the model       (A)
    lib/agent/                planner, tools, prompts, mockPlan.ts     (A)
    lib/booking/              booking pipeline                         (A)
    lib/supabase/             clients                                  (B)
    lib/calendar/             Google Calendar helpers                  (B)
    supabase/schema.sql       tables and security rules                (B)
    docs/contract.md          the contract from section 5              (both)
    .env.example              variable names only, never values        (both)
  demo/
    seed/demo-plan.json       pre-tested plan for the live demo
    script.md                 what to say and click
    devpost.md                submission text
    screenshots/
```

- Edit your own folders. The letters show who owns what, which keeps merge conflicts rare. Tell each other in chat before touching package.json, schema.sql or types.ts.
- Pull before you start, commit small, push at least every 30 to 45 minutes. Terminal: git add ., git commit -m "message", git push.
- Work on the main branch. If a conflict appears, stop and resolve it together in VS Code's merge editor. Do not force push.
- Keys live in .env (git ignores it) and in Vercel environment variables. A key pushed to a public repo is scraped within minutes; if it happens, rotate the key immediately.
- Deploy on every push and check the live URL. Judges try the live app, not your laptop.

11. Credits and accounts checklist

From the host's message in Discord. Codes are in that message. Start the ones that need a DM, because they involve waiting. The credits are in your accounts, so share the keys with your teammate through a private message, never in the repo or a public channel.

Service	Used for	How to get it	Done
Omnara	The agent's model	Create an account at app.omnara.com and DM kart your organization ID. Ask how to get the API key or endpoint	[ ]
Supabase	Database, auth, realtime	DM kart, then create the project	[ ]
context.dev	Venue page details	Redeem on the redeem-credits page with the Discord code	[ ]

Service	Used for	How to get it	Done
Exa	Venue search	Not on the all-attendee list. Ask in Discord or check free signup credits. Fallback: context.dev plus the model's own web search	[ ]
Google Cloud	Login and Calendar	OAuth app in Testing mode, calendar scope, test users	[ ]
Freestyle or Unikraft	Sandbox for the browser agent	Redeem when you reach rung 5	[ ]
Vercel, Resend	Hosting, email fallback	Free accounts	[ ]
Stripe, Primitive	Not in the plan	Skip unless ahead of schedule	[ ]

12. Timeline

Assumes you start around 1:00. If you start later, shift the middle blocks but keep the two fixed points: feature freeze at 4:15 and submission by 5:30.

Time	A: the agent (teammate)	B: the web app (you)
1:00 to 1:45	types.ts, mockPlan.ts, llm.ts connected to the Omnara key, test one Exa and one context.dev call, write contract.md with B	Next.js in main, deploy to Vercel, Supabase project, schema.sql (profiles, plans), Google OAuth with calendar scope and test users
1:45 to 2:45	Real planning v1 against a sample profile, /api/plan route running the mock end to end	Login, profile form, Plan my day button, timeline rendering the mock plan
2:45 to 3:30	Swap the mock for the real agent; validation and progress messages; start booking tiers 1, 3, 5	Progress display, locked and revealed stops, demo mode, google_tokens saved at login
3:30 to 4:15	Test bookings on real venues; browser agent attempt on one platform; re-plan if time	Calendar events created and patched, friends as attendees, booking chips, polish on phone
4:15	Feature freeze	Feature freeze
4:15 to 5:15	Full run on the live URL, fix failures, backup video, Devpost text	Same; test on phones; rehearse the demo twice
5:15 to 5:30	Submit with buffer, not at 5:29	Submit with buffer

13. Demo plan

- Pre-test one plan and keep it in demo/seed/demo-plan.json, using only venues whose booking path worked.
- Live flow: log in, show the profile, tap Plan my day and let the judges watch the progress messages, show the teaser and the calendar placeholder, then flip demo mode on and watch two or three stops reveal while the calendar event updates.
- Show a confirmed booking with its proof chip.
- The line for judges: the agent does real things in the real world, and it is live on the internet right now.

- Record one clean run as a backup video and rehearse twice on the production URL.

14. Risks and mitigations

Risk	Mitigation
The agent is slower than a web request allows	Row-and-status design, longer maxDuration, progress messages so the wait feels intentional
A and B block each other	Contract plus mock plan in the first 30 minutes; each builds against it independently
Omnara credits or key delayed	Everything goes through lib/llm.ts; point it at another provider temporarily
Hallucinated or closed venues	Every stop must come from a search result and pass the hours and travel check before it is saved
Booking blocked or flaky	Tiered fallbacks, demo limited to tested venues, walk-in swap
Calendar login missing the scope	Request it in the first version of the login; save the refresh token at sign-in
Broken main branch or leaked keys	Own your folders, pull before work, .env ignored from the first commit
Scope creep	Follow the rungs; freeze at 4:15